

基于 Transformer 的结构复现与位置编码、残差连接消融实验研究

摘要 (Abstract)

Transformer 是 Vaswani 等人在 Attention Is All You Need 中提出的经典序列建模架构。该模型完全舍弃循环神经网络和卷积神经网络，仅依靠自注意力机制建模序列中不同位置之间的依赖关系，在机器翻译任务中取得了显著效果。本文首先阅读并复现 Transformer 的核心结构，包括 encoder-decoder 架构、scaled dot-product attention、multi-head attention、position-wise feed-forward network、positional encoding、residual connection 与 layer normalization 等模块。考虑到原论文在 WMT 2014 机器翻译任务上的完整复现实验需要较高算力，本文采用轻量级合成序列反转任务作为复现与验证平台。该任务要求模型将输入整数序列映射为其反转序列，能够有效检验模型是否理解 token 顺序以及输入输出位置之间的对应关系。

在此基础上，本文进一步围绕 Transformer 中两个关键结构进行消融实验：第一，比较无位置编码、简单绝对位置编码、可学习位置编码和原论文 sinusoidal 位置编码的影响；第二，比较标准 Add & Norm、去掉残差连接、去掉 LayerNorm 以及同时去掉二者时模型训练和泛化性能的变化。初步实验表明，在不加入位置编码时，模型的训练 loss 和验证 loss 虽然持续下降，但完整序列准确率几乎始终为 0，说明仅依靠 self-attention 难以完成顺序敏感的序列映射任务。该结果验证了位置编码在 Transformer 中提供顺序信息的核心作用。后续通过完整实验结果进一步分析残差连接对训练稳定性和梯度传播的影响。

关键词: Transformer; Self-Attention; Positional Encoding; Residual Connection; Ablation Study; Sequence-to-Sequence

1. 引言 (Introduction)

序列建模是模式识别与机器学习中的重要问题，广泛应用于机器翻译、语音识别、语言建模和时间序列预测等任务。传统序列转导模型通常依赖循环神经网络、长短期记忆网络或卷积神经网络来建模序列内部依赖。然而，循环结构存在明显的顺序计算限制，难以在序列维度上充分并行化；卷积结构虽然可以并行计算，但若建模远距离 token 之间的关系，往往需要堆叠多层网络。

Vaswani 等人在 Attention Is All You Need 中提出 Transformer 架构，核心思想是完全去除 recurrence 和 convolution，仅依靠 attention mechanism 建模输入与输出序列之间的依赖关系。论文指出，Transformer 能够显著提高并行化程度，并在 WMT 2014 英德、英法机器翻译任务上取得优良结果。原论文中 Transformer big model 在 WMT 2014 English-to-German 任务上达到 28.4 BLEU，在 English-to-French 任务上达到 41.8 BLEU。

本文的工作包括两部分。第一部分复现 Transformer 的核心模型结构；第二部分基于复现模型进行模块消融实验，重点研究位置编码和残差连接两个结构的必要性。由于原论文完整机器翻译实验训练成本较高，原论文 base model 训练需要约 100,000 steps，big model 训练需要约 300,000 steps，且使用多张 P100 GPU。因此，本文参考 Tensor2Tensor 的 algorithmic reverse problem 设置，构造了一个轻量级 reverse sequence-to-sequence 数据生成器，以较低计算成本验证 Transformer 模块设计的合理性。

2. Transformer 原理与结构复现

2.1 Transformer 总体架构

Transformer 采用 encoder-decoder 架构。Encoder 将输入序列 $(x_1, x_2, \dots, x_n)$ 映射为连续表示 $(z_1, z_2, \dots, z_n)$ ，decoder 再基于 encoder 输出逐步生成目标序列 $(y_1, y_2, \dots, y_m)$ 。在生成第 i 个 token 时，decoder 只能使用已生成的前缀信息，因此该过程是自回归的。原论文中的 encoder 和 decoder 均由多层相同结构堆叠而成，其中 encoder layer 包含 multi-head self-attention 和 position-wise feed-forward network 两个子层；decoder layer 在此基础上额外加入 encoder-decoder attention 子层。

本文复现了该 `encoder-decoder` 主体结构。为适应 CPU 训练条件，本文没有完全使用原论文的 $N = 6, d_{model} = 512, h = 8, d_{ff} = 2048$ 大配置，而是采用较小的轻量配置，例如 $N = 4, d_{model} = 128, h = 4, d_{ff} = 512$ 。该设置保留了 Transformer 的关键结构，同时显著降低训练成本。

2.2 Scaled Dot-Product Attention

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

在本文实现中，`MultiHeadAttention` 模块首先将输入映射为 `Q,K,V`，再计算 `scaled attention score`，并在需要时加入 `mask`，最后通过 `softmax` 得到注意力权重并对 `V` 加权求和。代码中还保存了最后一层 `decoder-source attention` 权重，用于后续生成 `attention heatmap`。

单头 attention 只能在一个表示空间中计算注意力关系,而 multi-head attention 将 O, K, V 分别投影到多个子空间中并行计算 attention。其形式为:

$$MultiHead(Q, K, V) = Concat(head_1, \dots, head_h)W^O$$

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$$

原论文认为，多头注意力使模型能够在不同表示子空间和不同位置上同时关注信息，从而提升表达能力。本文实现中使用 4 个 attention heads，每个 head 的维度为 d_{model}/h 。对于序列反转任务而言，multi-head attention 的作用可以理解为：不同 head 可能学习不同形式的位置对齐关系，例如目标序列第一个 token 应关注源序列最后一个 token，目标序列第二个 token 应关注源序列倒数第二个 token。

2.4 Position-wise Feed-Forward Network

除 attention 子层外，Transformer 的每个 encoder 和 decoder layer 都包含一个 position-wise feed-forward network:

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

该模块对序列中每个位置独立且共享参数地进行非线性变换。原论文中 $d_{model} = 512, d_{ff} = 2048$ 。本文在轻量复现中采用较小维度，例如 $d_{model} = 128, d_{ff} = 512$ ，以降低训练开销。

2.5 Positional Encoding

由于 Transformer 不使用 RNN 或 CNN，模型本身无法天然感知 token 的先后顺序。因此，原论文在 encoder 和 decoder 底部加入 positional encoding，将位置信息注入 token embedding 中。原论文指出，为了让模型利用序列顺序，必须向模型输入中加入关于相对或绝对位置的信息。

原论文采用正弦和余弦函数构造位置编码：

$$\begin{aligned} PE(pos, 2i) &= \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \\ PE(pos, 2i+1) &= \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right) \end{aligned}$$

这种 sinusoidal positional encoding 不需要额外学习参数，并且论文认为它可能帮助模型外推到训练中未出现过的更长序列。原论文也提到，learned positional embeddings 与 sinusoidal positional encoding 在实验中结果接近，但最终选择 sinusoidal 的原因是其可能具备更好的长度外推能力。

本文复现了 sinusoidal positional encoding，并额外实现三种对照方式：不加

入位置编码、简单绝对位置编码和可学习位置编码，以完成位置编码消融实验。

2.6 Residual Connection 与 Layer Normalization

原论文中，每个子层外部都使用 residual connection，并在残差相加后进行 layer normalization：

$$\text{LayerNorm}(x + \text{Sublayer}(x))$$

其中 $\text{Sublayer}(x)$ 可以是 multi-head attention 或 feed-forward network。Residual connection 可以缓解深层网络训练中的梯度传播问题，使输入信息可以通过捷径直接传递到后续层；LayerNorm 则有助于稳定不同层之间的激活分布。原论文明确指出 encoder 和 decoder 中每个子层都采用 residual connection followed by layer normalization。

本文在 SublayerConnection 中复现了标准 Add & Norm 结构，并进一步设置了去掉 residual、去掉 LayerNorm 以及同时去掉二者的消融版本，以分析该模块对训练稳定性和性能的影响。

3 实验设计

3.1 实验任务：Synthetic Reverse Task

本文没有直接复现 WMT 2014 机器翻译任务，而是参考 Tensor2Tensor 中的 algorithmic reverse problem 设置，构造了一个轻量级的 reverse sequence-to-sequence 数据生成器^[3]。与 Tensor2Tensor 的 AlgorithmicReverseBinary40 / AlgorithmicReverseDecimal40 类似，输入为随机符号序列，目标输出为其反转序列。^[7]例如：

src = [5,8,2,9]

tgt = [9,2,8,5]

在实际代码中，目标序列还加入了 BOS 和 EOS 标记，用于表示生成开始和结束。数据集通过 ReverseDataset 随机生成，训练集、验证集和测试集长度范围为 5 到 20；另外设置 OOD 测试集，长度范围为 21 到 40，用于观察模型对更长序列的外推能力。^{[4][5]}本文代码中定义了 PAD=0、BOS=1、EOS=2，并通过 padding 将同一 batch 内样本对齐。

选择 reverse task 的原因是：该任务对顺序信息高度敏感。若模型无法区分

token 的位置，即使知道输入中包含哪些 token，也无法正确生成反转后的序列。因此，该任务特别适合研究位置编码的必要性^[6]。

3.2 模型实现

本文基于 PyTorch 重新整理了一个轻量级 Transformer 实验框架。代码包含以下模块：

模块	功能
ReverseDataset	构造序列反转数据
TokenEmbedding	将 token 映射到连续向量空间
NoPositionalEncoding	无位置编码对照组
SimpleAbsolutePositionalEncoding	简单绝对位置编码
LearnedPositionalEncoding	可学习位置编码
SinusoidalPositionalEncoding	原论文正弦/余弦位置编码
MultiHeadAttention	多头注意力
FeedForward	位置前馈网络
SublayerConnection	Add & Norm 及其消融版本
EncoderLayer / DecoderLayer	Transformer 编码器和解码器层
Seq2SeqTransformer	完整 encoder-decoder 模型

代码默认使用 `--mode all`，会依次运行位置编码实验和残差结构实验。实验组由 `get_experiments()` 控制，包括 `pe_none`、`pe_simple`、`pe_learned`、`pe_sinusoidal` 以及残差连接相关对照组。代码同时会保存每组实验的 `history` 文件、最终 `summary.csv`、验证 `loss` 曲线和 `baseline attention heatmap`。

3.3 实验环境

本文实验在本地 Windows 环境下完成，使用 Python 与 PyTorch 实现。由于原 Harvard annotated-transformer 仓库的 `requirements.txt` 固定了较旧版本的 `torch`、`torchtext`、`spacy` 等依赖，直接安装可能与当前 Python 版本不兼容。因此，本文没有完全依赖原仓库环境，而是将核心模型结构重新整理到独立的 PyTorch 项目中。实际使用的核心依赖包括：

依赖	用途
Python 3.11	运行环境
PyTorch	模型构建与训练
NumPy	随机数据生成与数值计算
Pandas	保存和读取实验结果
Matplotlib	绘制 loss 曲线和 attention heatmap
tqdm	显示训练进度

3.4 训练设置

默认训练参数如下：

参数	默认值
训练集大小	12000
验证集大小	1000
测试集大小	1000
训练长度范围	5–20
OOD 长度范围	21–40
d_{model}	128
attention heads	4
encoder/decoder layers	4
d_{ff}	512
dropout	0.1
batch size	128
epochs	12
optimizer	Adam
warmup steps	400
gradient clipping	5.0

优化器采用 Adam，参数设置参考 Transformer 原论文中常用形式，即 $\beta_1 = 0.9, \beta_2 = 0.98, \epsilon = 10^{-9}$ 。学习率采用 Transformer 风格 warmup schedule：

$$lr = d_{model}^{-0.5} \cdot \min(\text{step}^{-0.5}, \text{step} \cdot \text{warmup}^{-1.5})$$

训练过程中使用 teacher forcing，即 decoder 输入为目标序列右移一位后的前缀，预测目标为真实序列的下一位 token。代码中每个 epoch 后会在验证集上计算 loss、teacher token accuracy、greedy token accuracy 和 sequence accuracy。

3.5 评价指标

本文使用以下指标评价模型性能：

指标	含义
Train Loss	训练集交叉熵损失
Valid Loss	验证集交叉熵损失
Teacher Token Accuracy	在 teacher forcing 条件下逐 token 预测准确率
Greedy Token Accuracy	自回归 greedy decoding 条件下逐 token 预测准确率
Sequence Accuracy	完整序列完全预测正确的比例
Gradient Norm	梯度范数，用于观察训练稳定性
Test-ID Accuracy	与训练长度相同分布的测试集表现
Test-OOD Accuracy	更长序列上的长度外推表现

其中，sequence accuracy 是最严格的指标。对于 reverse task，只有整条反转序列每个 token 都正确，才计为一个正确样本。因此，sequence accuracy 更能反映模型是否真正学会了序列级映射关系。

4 消融实验设计

4.1 位置编码实验

位置编码实验固定残差结构为标准 Add & Norm，仅改变 positional encoding 类型。实验组如下：

实验组	位置编码	残差结构	目的
No PE	none	standard	验证没有位置信息时模型表现
Simple PE	simple absolute	standard	验证简单绝对位置编码效果
Learned PE	learned	standard	验证可学习位置编码效果
Sinusoidal PE/Baseline	sinusoidal	standard	原论文位置编码，作为 baseline

该实验对应作业要求 2.1。核心问题是：Transformer 是否必须加入位置编码？如果加入不同形式的位置编码，模型在顺序敏感任务上的表现有何差异？

4.2 残差连接实验

残差结构实验固定位置编码为 sinusoidal positional encoding，仅改变子层连接方式。实验组如下：

实验组	子层结构	目的
Standard Add & Norm	LayerNorm(x+Sublayer(x))	原论文标准结构
No Residual	LayerNorm(Sublayer(x))	去掉残差连接
No LayerNorm	x+Sublayer(x)	去掉 LayerNorm
No Residual & No LayerNorm	Sublayer(x)	同时去掉二者

该实验对应作业要求 2.3。核心问题是：Transformer 中 residual connection 是否只是辅助结构，还是对模型训练具有关键作用？LayerNorm 对训练稳定性是否同样重要？

5 实验结果与分析

5.1 位置编码实验分析

5.3.1 不加位置编码实验结果分析

pe_none 实验组的实验设置为 $PE = none$, $residual = standard$ ，即没有位置信息。该实验结果如下：

epoch	train_loss	valid_loss	valid_teacher_token_acc	valid_greedy_token_acc	valid_sequence_acc	grad_norm
1	3.18	2.62	16.87	14.73	0.00	1.41
2	2.55	2.27	25.32	14.10	0.00	1.43
3	2.35	2.15	27.98	14.77	0.10	1.41
4	2.24	2.12	28.29	14.50	0.00	1.17
5	2.15	1.99	30.47	14.91	0.00	0.94
6	2.04	1.92	31.90	14.70	0.00	0.83
7	1.97	1.84	32.68	15.48	0.00	0.75
8	1.92	1.84	32.48	15.28	0.20	0.73
9	1.89	1.81	32.75	14.18	0.10	0.71
10	1.86	1.76	34.00	15.05	0.10	0.69
11	1.83	1.75	33.57	15.23	0.00	0.67
12	1.81	1.74	34.00	15.52	0.00	0.66

在不加入位置编码的情况下，模型的训练损失从 3.18 下降到 1.81，验证损失从 2.62 下降到 1.74，说明模型仍然能够学习一部分 token 分布规律。然而，模型的 greedy token accuracy 始终维持在约 14%–16%，完整序列准确率几乎始终为 0，最高仅为 0.20%。这表明模型虽然在 teacher forcing 条件下具备一定的局部 token 预测能力，但在真实自回归生成时无法稳定完成序列反转任务。该结果说明，没有位置编码时，Transformer 难以区分 token 的顺序位置，因而无法学习输入序列与反转输出序列之间的精确位置对应关系。

因此，不加位置编码实验支持以下结论：位置编码并不是 Transformer 中可有可无的附加模块，而是 self-attention 处理有序序列任务的必要信息来源。它的核心作用是向模型注入 token 的绝对位置、相对位置或顺序方向信息，使 attention 机制能够在内容相关性之外进一步建模位置关系。

5.3.2 简单位置编码实验结果分析

简单位置编码训练效果如下：

Epoch	Train Loss	Valid Loss	Teacher Token Acc (%)	Greedy Token Acc (%)	Sequence Acc (%)	Grad Norm
1	3.20	2.62	17.79	15.42	0.00	1.35
2	2.59	2.35	23.72	14.10	0.00	1.46
3	2.38	2.21	26.54	14.42	0.00	1.42
4	2.27	2.19	26.65	14.07	0.00	1.19
5	2.17	1.99	30.99	14.93	0.00	0.96
6	2.04	1.91	32.35	15.03	0.00	0.85
7	1.83	1.39	48.21	23.33	0.30	1.11
8	1.48	1.01	62.78	35.20	3.80	1.54
9	1.20	0.69	74.46	45.99	8.70	1.97

Epoch	Train Loss	Valid Loss	Teacher Token Acc (%)	Greedy Token Acc (%)	Sequence Acc (%)	Grad Norm
10	0.98	0.65	77.02	49.35	14.00	2.39
11	0.79	0.40	86.76	59.93	27.00	2.74
12	0.65	0.41	86.88	56.27	27.10	3.01

简单绝对位置编码实验结果如表所示。与不加位置编码相比，简单绝对位置编码显著改善了模型的训练效果。模型的训练损失从 3.20 下降到 0.65，验证损失从 2.62 下降到 0.41；teacher forcing 条件下的 token accuracy 从 17.79% 提升到 86.88%，greedy decoding 条件下的 token accuracy 从 15.42% 提升到 56.27%。完整序列准确率也从 0% 提升到 27.10%。这说明即使是较简单的绝对位置编码，也能够为 Transformer 提供必要的顺序信息，使模型具备一定的序列反转能力。

但是，简单绝对位置编码的效果仍然明显低于原论文的 sinusoidal 位置编码。它虽然能告诉模型每个 token 所处的绝对位置，但表达形式较为简单，对复杂位置关系和长度外推的建模能力有限。因此，该实验说明：位置编码本身是必要的，但位置编码的形式也会显著影响 Transformer 对顺序关系的学习能力。

5.3.3 可学习位置编码实验结果分析

Epoch	Train Loss	Valid Loss	Teacher Token Acc (%)	Greedy Token Acc (%)	Sequence Acc (%)	Grad Norm
1	3.17	2.57	18.13	15.02	0.00	1.44
2	2.55	2.25	25.98	14.76	0.00	1.41
3	2.34	2.12	30.29	16.30	0.00	1.46
4	1.99	1.35	53.49	29.22	1.20	1.93
5	1.44	1.09	64.44	41.83	4.40	2.20
6	1.03	0.62	79.20	49.20	17.70	2.33
7	0.71	0.33	89.15	61.23	36.20	2.25
8	0.48	0.18	94.37	74.30	52.30	1.89
9	0.31	0.11	96.67	78.93	64.50	1.55
10	0.22	0.08	97.46	83.33	70.60	1.30
11	0.16	0.07	97.58	83.00	71.70	1.08
12	0.14	0.06	98.05	86.75	76.90	1.00

可学习位置编码实验结果如表所示。模型的训练损失从 3.17 下降到 0.14，验证损失从 2.57 下降到 0.06，表明模型在训练集和验证集上均实现了稳定收敛。在准确率方面，teacher forcing 条件下的 token accuracy 从 18.13% 提升到 98.05%，greedy decoding 条件下的 token accuracy 从 15.02% 提升到 86.75%，完整序列准确率从 0% 提升到 76.90%。这说明可学习位置编码能够有效提供位

置信息，使 Transformer 学会序列反转任务中的输入输出位置对应关系。

与简单绝对位置编码相比，可学习位置编码的效果显著更好。简单绝对位置编码最终 sequence accuracy 为 27.10%，而可学习位置编码达到 76.90%，说明位置编码的表达能力会直接影响 Transformer 对顺序关系的学习能力。可学习位置编码通过为每个位置分配独立可训练向量，使模型能够更灵活地建模不同位置之间的对应关系。

但是，可学习位置编码仍然略弱于 sinusoidal 位置编码 baseline。Sinusoidal PE 的最终验证集 sequence accuracy 约为 85.50%，高于可学习位置编码的 76.90%。此外，可学习位置编码通常更依赖训练时出现过的位置范围，对于训练长度之外的位置，其向量可能没有得到充分训练，因此长度外推能力可能弱于 sinusoidal PE。整体来看，可学习位置编码在训练长度范围内表现较强，但在泛化到更长序列时可能存在局限。

5.3.4 余弦/正弦位置编码实验结果分析

论文正弦/余弦位置编码实验结果如下：

Epoch	Train Loss	Valid Loss	Teacher Token Acc (%)	Greedy Token Acc (%)	Sequence Acc (%)	Grad Norm
1	3.24	2.64	17.39	15.32	0.00	1.54
2	2.48	1.73	40.35	27.03	0.10	1.85
3	1.76	0.80	73.23	47.99	10.70	2.83
4	1.08	0.39	87.41	57.72	28.90	2.67
5	0.61	0.17	95.16	76.03	53.40	2.08
6	0.37	0.12	96.47	79.87	62.20	1.65
7	0.27	0.13	96.44	78.36	63.70	1.36
8	0.20	0.07	97.97	88.77	76.30	1.15
9	0.16	0.06	98.36	89.24	79.80	1.00
10	0.14	0.07	98.13	87.35	77.40	0.97
11	0.12	0.03	98.91	92.54	86.60	0.85
12	0.10	0.04	98.80	92.52	85.50	0.78

原论文正弦/余弦位置编码实验结果如表所示。模型的训练损失从 3.24 下降到 0.10，验证损失从 2.64 下降到 0.04，说明模型在训练集和验证集上均实现了稳定收敛。在准确率方面，teacher forcing 条件下的 token accuracy 从 17.39% 提升到 98.80%，greedy decoding 条件下的 token accuracy 从 15.32% 提升到 92.52%，完整序列准确率从 0% 提升到 85.50%，并在第 11 个 epoch 达到最高值 86.60%。

与无位置编码、简单绝对位置编码和可学习位置编码相比，sinusoidal PE 的表现最好。无位置编码最终 sequence accuracy 为 0.00%，简单绝对位置编码为 27.10%，可学习位置编码为 76.90%，而 sinusoidal PE 达到 85.50%。这说明 Transformer 不仅需要位置编码来感知 token 顺序，而且位置编码的表达形式会直接影响模型学习顺序关系的能力。

Sinusoidal PE 的优势可能来自其多频率正弦/余弦函数表示。不同维度编码不同尺度的位置变化，使模型能够同时捕捉局部位置差异和较长距离的位置关系。对于序列反转任务而言，模型必须学习输入位置与输出反向位置之间的对应关系，因此高质量的位置表示尤为重要。实验结果表明，原论文采用的 sinusoidal positional encoding 能够有效增强 Transformer 的顺序建模能力，是本实验中表现最优的位置编码方式。

5.2 残差连接消融结果

实验固定采用原论文 sinusoidal positional encoding，仅改变 Transformer 子层外部的连接结构。残差连接消融实验的最终结果对比如下：

实验组	Train Loss	Valid Loss	Teacher Token Acc (%)	Greedy Token Acc (%)	Sequence Acc (%)	Grad Norm
Standard Add & Norm	0.10	0.04	98.80	92.52	85.50	0.78
No Residual	2.88	2.84	12.44	12.44	0.00	0.24
No LayerNorm	104.96	60.59	5.49	5.33	0.00	62596.65
No Residual & No LayerNorm	3.06	3.04	11.18	11.18	0.00	0.55

表中数据显示，标准 Add & Norm 结构取得了最优结果，其最终训练损失为 0.10，验证损失为 0.04，teacher token accuracy 达到 98.80%，greedy token accuracy 达到 92.52%，完整序列准确率达到 85.50%，并在第 11 个 epoch 达到最高 sequence accuracy 86.60%。这说明在保留残差连接和 LayerNorm 的情况下，Transformer 能够稳定学习 reverse task 中的输入输出位置对应关系。

当去掉残差连接后，模型性能显著下降。No Residual 组最终验证损失为 2.84，teacher token accuracy 和 greedy token accuracy 均仅为 12.44%，完整序列

准确率始终为 0。该结果说明，残差连接对 Transformer 的信息传递和梯度传播非常重要。残差路径允许每个子层学习对输入表示的增量修正，同时保留低层信息；去掉该结构后，模型难以稳定学习序列映射关系。

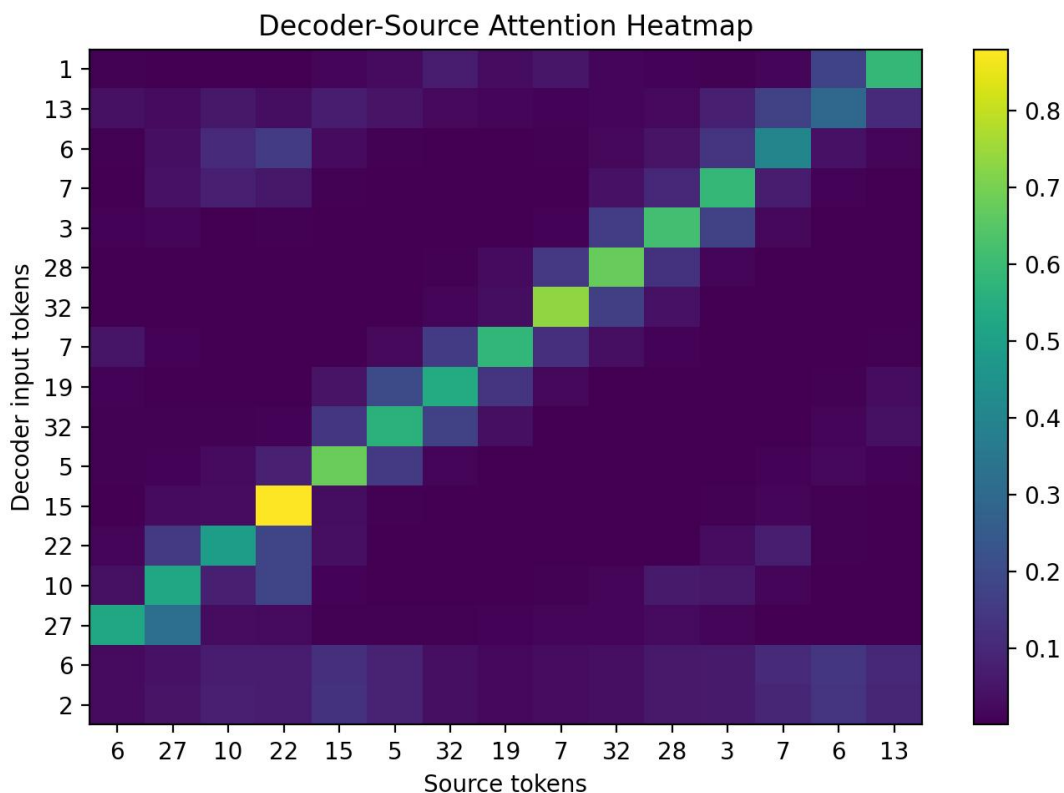
当去掉 LayerNorm 后，模型训练出现明显不稳定。No LayerNorm 组最终训练损失高达 104.96，验证损失为 60.59，完整序列准确率为 0，梯度范数达到 62596.65，远高于标准结构的 0.78。这说明 LayerNorm 对稳定 Transformer 的激活分布和梯度规模具有关键作用。没有 LayerNorm 时，残差路径与子层输出不断叠加，可能导致数值尺度失控，从而引发训练震荡或梯度爆炸。

当同时去掉残差连接和 LayerNorm 后，模型同样无法学习任务。No Residual & No LayerNorm 组最终验证损失为 3.04，greedy token accuracy 为 11.18%，sequence accuracy 为 0。虽然该组没有出现像 No LayerNorm 那样的梯度爆炸，但模型几乎停留在无效学习状态，说明缺少残差路径和归一化机制后，Transformer 难以形成有效的深层表示。

综上，残差连接和 LayerNorm 是 Transformer 稳定训练不可或缺的结构。残差连接主要保证信息和梯度的有效传递，LayerNorm 主要保证层间数值分布稳定。二者共同构成 Add & Norm 模块，使深层 attention 网络能够有效优化。本实验结果表明，Transformer 的有效性不仅来自 self-attention 本身，也依赖于 residual connection、LayerNorm、位置编码和前馈网络等结构的协同作用。

5.3 Attention 可视化分析

为了进一步观察 Transformer 是否真正学习到了序列反转任务中的位置对应关系，本文对 baseline 模型的 decoder-source attention 进行了可视化分析。该 baseline 模型采用原论文的 sinusoidal positional encoding，并保留标准 Add & Norm 结构。可视化结果如图所示。



横轴表示输入序列中的 source tokens，纵轴表示 decoder 输入 tokens，颜色越亮表示 attention 权重越大。

从图中可以看到，attention 权重整体呈现出较明显的反对角线分布。这与序列反转任务的目标是一致的。对于 reverse task，模型需要学习如下映射关系：

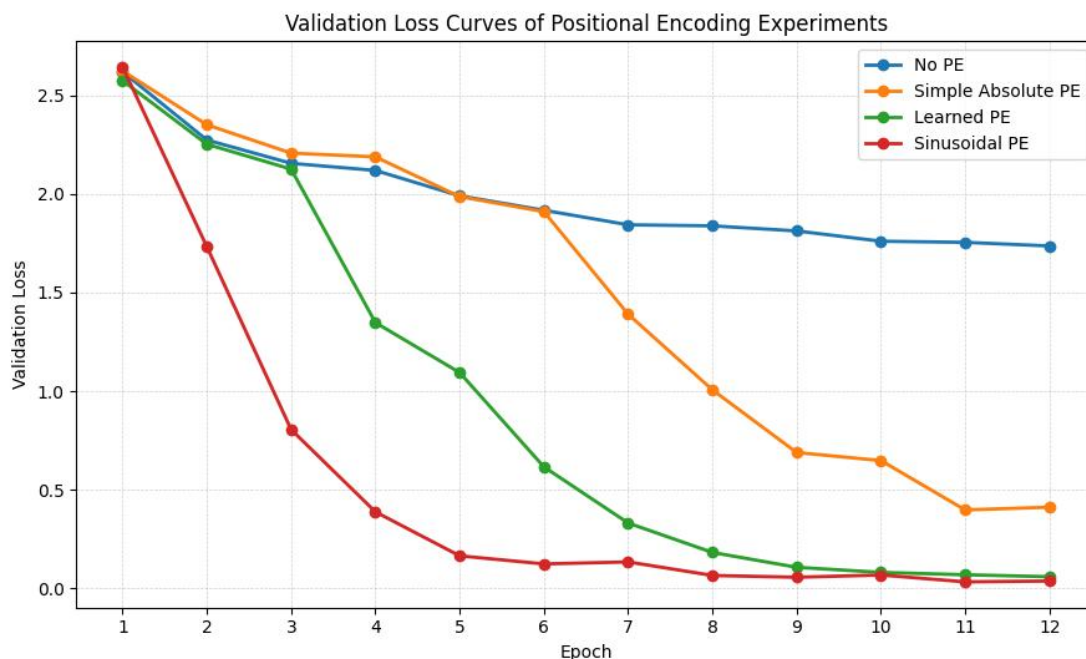
$$src = [x_1, x_2, \dots, x_n]$$

$$tgt = [x_n, x_{n-1}, \dots, x_1]$$

因此，当 decoder 生成目标序列的前几个 token 时，应主要关注 source 序列靠后的位置；当 decoder 继续向后生成时，注意力关注区域应逐步向 source 序列前部移动。图中的高亮区域正是从右上方向左下方移动，说明模型在生成目标 token 时倾向于关注输入序列中对应的反向位置。

Attention 可视化进一步支持了本文前面的实验结论：位置编码为 Transformer 提供了必要的顺序信息，使 self-attention 和 encoder-decoder attention 能够学习位置相关的序列映射；而标准 Add & Norm 结构保证了模型能够稳定训练，从而形成清晰的注意力对齐关系。

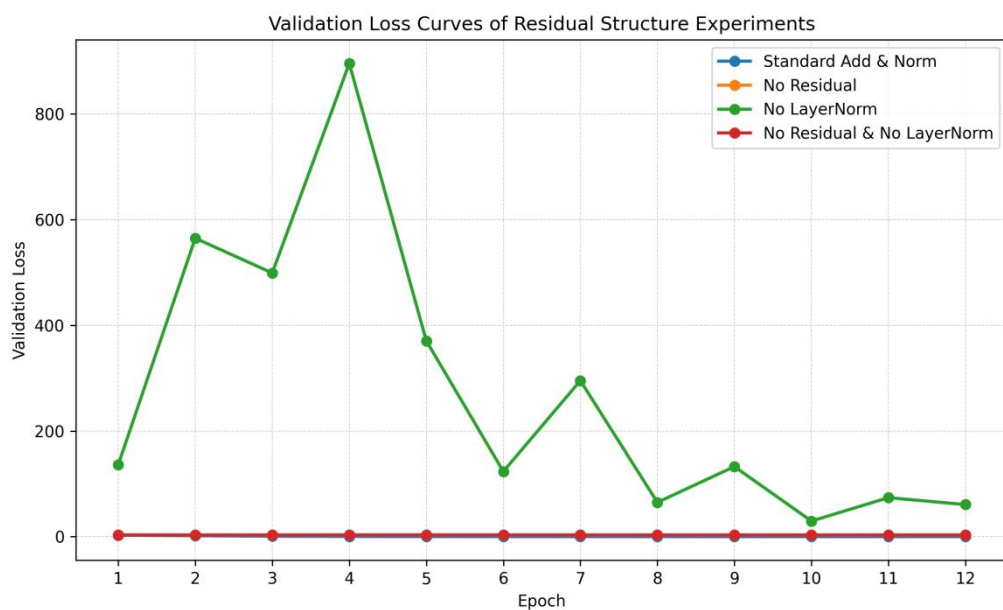
不同位置编码方式下的验证集损失曲线如下：



可以看到，Sinusoidal PE 的验证损失下降最快且最终最低；Learned PE 次之；Simple Absolute PE 有一定效果但收敛较慢；No PE 的验证损失下降有限，说明缺少位置编码时模型难以学习稳定的序列反转关系。

位置编码显著影响 Transformer 的收敛速度和最终性能。无位置编码时，验证损失始终停留在较高水平；简单绝对位置编码能够改善模型，但效果有限；可学习位置编码明显优于简单绝对位置编码；原论文的 Sinusoidal PE 收敛最快、最终验证损失最低，是本实验中表现最好的位置编码方式。

不同残差结构下的验证集损失曲线如下：



可以看到，标准 Add & Norm 的验证损失快速下降并保持在很低水平；No Residual 与 No Residual & No LayerNorm 的验证损失始终维持在较高水平；No LayerNorm 的验证损失出现剧烈波动并远高于其他实验组，表明去掉 LayerNorm 后训练极不稳定。

6 讨论

6.1 轻量级复现与原论文完整复现的区别

本文复现的是 Transformer 的核心模型结构，而不是原论文 WMT 2014 机器翻译结果的完全复现。原论文实验使用大规模机器翻译数据集，包括约 450 万英德句对和 3600 万英法句对，并使用多 GPU 训练。在课程作业场景下，直接复现原论文 BLEU 分数需要较高算力与较长训练时间。因此，本文选择 synthetic reverse task 作为可控实验任务，以较低成本验证 Transformer 的结构有效性和模块必要性。

这种轻量复现具有两个优点。第一，数据生成简单，不受复杂分词、语料清洗和大规模训练成本影响。第二，reverse task 对顺序关系高度敏感，非常适合研究位置编码和残差连接等模块的作用。然而，该任务也存在局限：它不能完全代表自然语言翻译任务的语义复杂性，因此本文结论主要针对序列顺序建模与结构消融，而不声称复现原论文的机器翻译性能。

6.2 位置编码的核心作用

不加位置编码实验表明，Transformer 即使 loss 下降，也可能无法完成严格的 sequence-level 任务。这说明在 self-attention 中，内容相关性和位置信息是两个不同层面的因素。Attention 可以回答“哪些 token 与当前 token 内容相关”，但如果没有位置编码，它无法回答“这些 token 位于哪里”。对于 reverse task，后一个问题完成任务的关键。

因此，位置编码的核心作用可以概括为三点：

第一，提供 token 的绝对位置，使模型能够区分相同 token 在不同位置出现的情况。第二，提供相对位置或方向线索，使模型能够学习“前后”“距离”“反转”等顺序关系。第三，在没有 recurrence 和 convolution 的情况下，补足 Transformer 对序列结构的感知能力。

这与原论文动机一致：Transformer 因为不含循环和卷积结构，所以必须通过 positional encoding 注入位置信息。

6.3 残差连接的预期作用

残差连接最初在 ResNet 中被提出，用于缓解深层网络的退化和梯度传播困

难问题。Transformer 虽然不是卷积网络，但 encoder 和 decoder 都由多个子层堆叠而成，同样存在深层训练困难。Add & Norm 结构使每个子层学习的是对输入表示的增量修正，而不是完全重新构造表示。这种设计可以保留低层信息，并为梯度提供更短的传播路径。

在本文残差消融实验中，如果 No Residual 或 No Residual & No LayerNorm 组明显劣于 Standard Add & Norm，则说明残差连接是 Transformer 稳定训练的重要组成部分。若 No LayerNorm 组出现 loss 波动或梯度范数异常，则说明 LayerNorm 对控制激活分布和训练稳定性具有关键作用。

7 结论

本文围绕 Attention Is All You Need 进行了结构复现与消融实验研究。首先，本文基于 PyTorch 复现了 Transformer 的核心 encoder-decoder 架构，包括 multi-head attention、scaled dot-product attention、position-wise feed-forward network、positional encoding、masked decoder self-attention、residual connection 与 layer normalization 等模块。其次，本文设计 synthetic reverse task 作为轻量级序列转导任务，在可控场景下验证 Transformer 的序列建模能力。最后，本文针对位置编码和残差连接两个关键模块进行消融实验。

当前已完成的不加位置编码实验显示：模型的 train loss 和 valid loss 均持续下降，teacher forcing 条件下 token accuracy 也有所提升，但 greedy decoding 下的 sequence accuracy 几乎始终为 0。这说明模型能够学习部分 token 分布规律，却无法稳定完成顺序敏感的完整序列反转任务。该结果有力说明，位置编码是 Transformer 处理有序序列任务的必要模块，其核心作用是向 self-attention 注入 token 的位置和顺序信息。

后续在完整实验结果基础上，本文将进一步比较 simple absolute PE、learned PE 和 sinusoidal PE 的差异，并分析 residual connection 与 LayerNorm 对模型收敛速度、泛化能力和梯度稳定性的影响。预计这些结果将进一步说明 Transformer 的高性能并非来自单一 attention 模块，而是来自 attention、位置编码、残差连接、归一化和前馈网络等多个组件的协同作用。

参考文献

- [1]Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. Attention Is All You Need. Advances in Neural Information Processing Systems, 2017.
- [2]Harvard NLP. The Annotated Transformer.本文参考其 Transformer 代码结构与注释说明，并在此基础上重新整理为独立的 PyTorch 消融实验代码。
- [3]Dehghani, M., Gouws, S., Vinyals, O., Uszkoreit, J., & Kaiser, Ł. Universal Transformers. ICLR, 2019.Transformer 类模型使用 Copy / Reverse / Addition 算法任务，报告 char-acc/seq-acc
- [4]Variš, D., & Bojar, O. Sequence Length is a Domain: Length-based Overfitting in Transformer Models. EMNLP, 2021. Transformer 对训练/测试长度分布差异敏感；reverse 是 string editing task
- [5]Bulatov, A., Kuratov, Y., & Burtsev, M. Recurrent Memory Transformer. NeurIPS, 2022.reverse task 用于测试完整序列信息保持和长程依赖；数据随机预生成
- [6]Zaremba, W., & Sutskever, I. Learning Simple Algorithms from Examples. ICML, 2016.reverse 是典型简单算法学习任务，需要按反方向输出序列
- [7]TensorFlow. Tensor2Tensor Algorithmic Data Generators: AlgorithmicReverseBinary40/AlgorithmicReverseDecimal40.AlgorithmicReverseBinary40 / Decimal40 的具体数据生成规则